Experiments: Transformer tokenization pipeline

Learning Objectives

- Understand why LLMs use subword tokenization.
- Learn how text is converted into tokens and token IDs.
- Explore attention masks, padding, and truncation.
- Prepare data for Transformer models such as BERT and GPT.

Experiment 1: Basic Hugging Face Tokenization

Objective
Learn how a Transformer tokenizer converts text into subword tokens.

Install

```
pip install transformers torch
```

Python Program

```
from transformers import AutoTokenizer

# Load BERT tokenizer
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

text = "Artificial Intelligence is transforming education."

# Tokenize
tokens = tokenizer.tokenize(text)

print("Original Text:")
print(text)

print("\nSubword Tokens:")
for i, token in enumerate(tokens):
    print(f"{i+1}. {token}")
```

Expected Output

```
Original Text:
Artificial Intelligence is transforming education.

Subword Tokens:
1. artificial
2. intelligence
3. is
4. transforming
5. education
6. .
```

Discussion

Students observe:

1. Normal words become tokens.
2. Punctuation is treated as separate tokens.
3. BERT converts text into lowercase.

Experiment 2: Token IDs and Vocabulary Mapping

Objective
Understand how tokens are converted into numbers.

Python Program

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

text = "Machine Learning is amazing."

tokens = tokenizer.tokenize(text)
ids = tokenizer.convert_tokens_to_ids(tokens)

print("{:<15}{}".format("Token","Token ID"))
print("-"*30)

for token,id in zip(tokens,ids):
   print("{:<15}{}".format(token,id))

print("\nDecoded Text:")
print(tokenizer.decode(ids))
```

Expected Output

```
Token          Token ID
------------------------------
machine        3698
learning       4083
is             2003
amazing        6429
.              1012
```

Discussion

Students learn

- Every token has a unique integer ID.
- LLMs process numbers instead of text.
- Vocabulary acts as a lookup table.

Experiment 3: Special Tokens, Padding and Attention Mask

Objective
Understand how Transformer inputs are prepared.

Python Program

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

sentences = [
   "Artificial Intelligence",
   "Deep Learning is a subset of Machine Learning."
]

encoding = tokenizer(
   sentences,
   padding=True,
   truncation=True,
   return_tensors="pt"
)

print("Input IDs\n")
print(encoding["input_ids"])

print("\nAttention Mask\n")
print(encoding["attention_mask"])

print("\nDecoded Sentences\n")

for ids in encoding["input_ids"]:
   print(tokenizer.decode(ids))
```

Sample Output

```
Input IDs

tensor([[ 101, 7976, 4454, 102,    0,    0,    0],
       [ 101, 2784, 4083, 2003, 1037, 4942, 1997, 3698, 4083, 1012,102]])

Attention Mask

tensor([[1,1,1,1,0,0,0],
       [1,1,1,1,1,1,1,1,1,1,1]])
```

Discussion
Students understand

- [CLS] token
- [SEP] token
- Padding
- Attention Mask
- Equal-length sequences

Experiment 4: Complete Transformer Input Pipeline
Objective
Prepare text exactly as a Transformer model receives it.
Python Program

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

text = "Transformers have revolutionized Natural Language Processing."

encoding = tokenizer(
   text,
   return_tensors="pt",
   padding="max_length",
   truncation=True,
   max_length=20
)

print("Original Text:\n")
print(text)

print("\nInput IDs:\n")
print(encoding["input_ids"])

print("\nAttention Mask:\n")
print(encoding["attention_mask"])

print("\nToken Type IDs:\n")
print(encoding["token_type_ids"])

print("\nTokens:\n")
print(tokenizer.convert_ids_to_tokens(
   encoding["input_ids"][0]
))
```

Sample Output

Input IDs
[101,19081,2031,4329,...,102,0,0,0]

Attention Mask
[1,1,1,1,1,1,1,1,1,1,0,0,0]

Token Type IDs
[0,0,0,0,0,0,0...]

Tokens
['[CLS]',
'transformers',
'have',
'revolution',
'##ized',
'natural',
'language',
'processing',
'.',
'[SEP]',
'[PAD]',
'[PAD]']

Discussion

Students observe

- Subword splitting (revolution + ##ized)
- [CLS] token for classification tasks
- [SEP] token marking the end of the sentence
- [PAD] tokens for fixed-length inputs
- token_type_ids for distinguishing sentence segments

Conceptual Workflow of Tokenization in Transformers

Raw Text

Tokenizer

Subword Tokens

Vocabulary Lookup

Token IDs

Special Tokens ([CLS], [SEP])

Padding & Truncation

Attention Mask

Tensor Format (PyTorch)

Transformer Model (BERT/GPT)

Learning Outcomes

After completing these experiments, students will be able to:

1. Explain why Transformer models use **subword tokenization** instead of simple word tokenization.
2. Convert text into **subword tokens** using the Hugging Face tokenizer.
3. Map tokens to **token IDs** and reconstruct text from IDs.
4. Understand the role of **special tokens** ([CLS], [SEP], [PAD]).
5. Explain the purpose of **attention masks** and **padding**.
6. Prepare text in the exact tensor format expected by **Transformer-based LLMs** such as BERT and GPT.

These four experiments provide a logical progression from basic tokenization to the complete input preprocessing pipeline used by modern Transformer-based language models, making them well suited for a postgraduate hands-on NLP workshop.